capital

Custom Security Plugin Development

Software Version 2015.1

February 2016

© 2016 Mentor Graphics Corporation
All rights reserved.
This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**
Ticket-Based User Authentication **3**
User Authentication Base Interfaces **5**
IXBaseLogin 5
IXBaseAuthenticationResponse 5
User Authentication Interfaces **6**
IXLogin 6
IXAuthenticationResponse 6
IXClientAuthenticationResponse 6
User Authentication Plugin Discovery **7**
User Authentication Examples **7**
Java class 7
User Access privileges Interfaces **10**
IXUserAccessibilityEvaluation 10
IXUserAccessibilityEvaluator 10
IXUserAccessibilityResult 11
UserAccessibility 11
User Access privileges Plugin working flow **12**
User Access privileges Examples **12**
Java class 12
Using User Authentication and User Access privileges plugin together **14**
Assumptions **14**

Introduction

This document describes how to develop a custom security plugin.

In order to support the diverse requirements of organizations, Capital has four options for user Authentication:

1. **Capital's in-built credential store**. Users login with their user name and password and these are validated against the credentials held in the Capital database.
2. **Single Sign On**. Once users have logged into Windows they are assumed to be authentic and Capital simply requests the user's name from the operating system and the user is not required to login to Capital.
3. **Delegated authentication**. Users login with their user name and password and these are sent encrypted to Capital Manager. They are then sent to an external system, e.g. LDAP or Active Directory, to be validated.
4. **Ticket-Based or 2-Tier Authentication**. Users login with their user name and password and these credentials are sent by Capital to an external system, e.g. Kerberos, to be validated. On successful authentication a ticket is issued by the external system and is sent to Capital Manager by the client. Capital Manager in turn sends this ticket to external system for validation.

Options 1 & 2 can be configured without the need to resort to custom coding. Options 3 & 4 require the development of custom plugins for the communication with the external authentication systems.

Although it is possible to externalize authentication, Capital continues to be responsible for authorization (roles and permissions). It is therefore essential that every Capital user who has an entry in the external authentication system will also have corresponding entry in Capital User. This is required to assign different permissions to the users whose authenticity has been verified by the authentication plugin. The super user's login credentials are always be stored in the Capital system. The Capital super user creates all the user information in Capital using the Capital User application. A password set for these users in Capital User is not used if pluggable authentication is used.

In addition to external authentication, individual **user's access** to Capital Projects and Designs can be controlled using the following options:

1. **Capital's in-built access restriction**. Using Capital Domains user's access to specific Designs and Projects can be restricted.
2. **Delegated access restriction** .In addition to the Domain support, a plugin can be used to restrict user's access to specific Designs and Projects based on conditions defined externally.

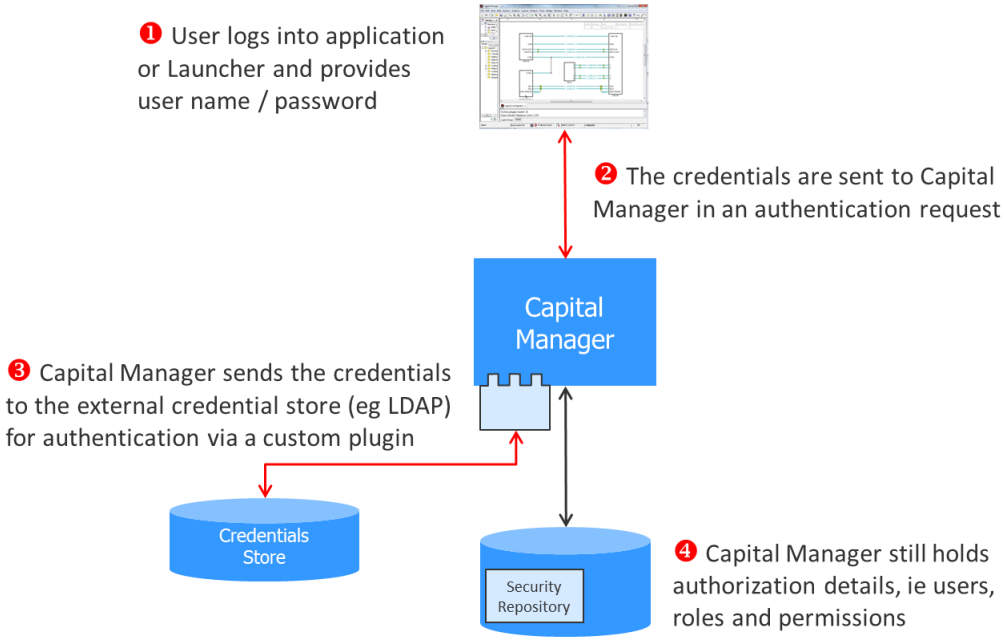
Delegated User Authentication.

Prior to using the delegated user authentication, the following set-up tasks are required:

- An authentication plugin should be written, say Login.jar. This plugin should implement the interface IXLogin (see Interfaces section).
- The jar (e.g. Login.jar) should be deployed in the plugins directory of the server machine where the Capital Manager application runs.
- Capital Manager should be restarted after deploying the plugin. On restart, Capital Manager scans the plugins directory and loads the authentication plugin. On successfully identifying authentication plugin, Capital Manager delegates the authentication to the plugin. Please note that if more than one

authentication plugins have been deployed, then all of them are ignored and user authentication is attempted using Capital's in-built credential store.

During the login, the events shown in the illustration below will take place:



Here is the main flow of events.

1) An application user starts the Capital Launcher or a Capital application. The user supplies the login name and password in the standard Capital login window.
2) Application or launcher makes an authentication request to Capital Manager and sends the encrypted credentials.
3) Because an authentication plugin has been loaded, Capital Manager passes the credentials to the plugin (calling the IXLogin.authenticate() method). This plugin is responsible for interacting with the external authentication system - it validates the credentials provided and responds back with an instance of IXAuthenticateResponse (see Interfaces section) to Capital Manager. If the user is authenticated, the Capital Manager communicates the success to the application. If the user fails to authenticate, then the corresponding error message, retrieved from the IXAuthenticateResponse object, is logged in the Capital Manager log.
4) The user's rights within Capital (roles, permissions and domains) are retrieved from Capital's database. These details are administered by Capital User.

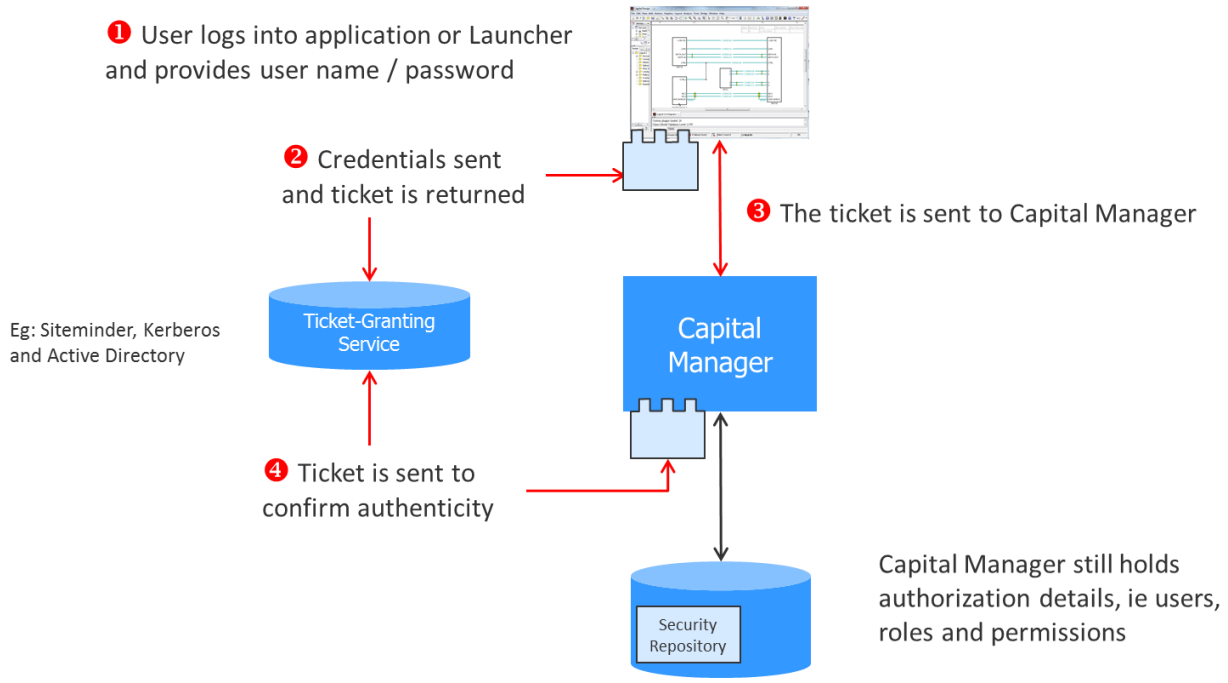
Ticket-Based User Authentication

Ticket-based authentication follows a similar pattern to delegated authentication – the main difference is that both the client and the server tiers make calls to the external service to check that the user is

authentic. User name and password are typically not transferred from client to server – but rather a ticket which has been issued by the external service (often known as a 'ticket-granting service').
Prior to using ticket-based authentication, the following set-up tasks are required:

- An authentication plugin should be written for the client which implements the interface IXClientLogin (see Interfaces section)
- The plugin's jar should be deployed in the plugins directory of the client machine where the Capital applications run. If one, **and only one**, implementation of IXClientLogin is found then it will be loaded by the applications on start up.
- An authentication plugin should be written for the server which implements the interface IXLogin (see Interfaces section). The jar containing this plugin is then deployed in the plugins directory of the server machine where the Capital Manager application runs.
- Capital Manager should be restarted after deploying the plugin. On restart, Capital Manager will scan the plugins directory and load the authentication plugin and recognises that authentication should be delegated. If more than one authentication plugins have been deployed, then all of the plugins are ignored and user authentication is attempted using Capital's in-built credential store.

During the login, the events shown in the illustration below will take place:



Here is the main flow of events.

1) An application user starts up the Capital Launcher or a Capital application. The user supplies the login name and password in the standard Capital login window.

2) Because an authentication plugin has been loaded, the application passes the credentials to the plugin (calling the IXClientLogin.authenticate() method). This plugin is responsible for interacting with the ticket-granting service and should the authentication succeed, the service provides a ticket which the plugin should process and return to the Capital application within an instance of IXClientAuthenticateResponse (see Interfaces section).

3) The Capital application or launcher makes an authentication request to Capital Manager and sends the IXClientAuthenticateResponse containing the ticket.

4) Because an authentication plugin has been loaded, Capital Manager passes the ticket to the plugin (calling the IXLogin.authenticate() method). The identifier and ticket held in IXClientAuthenticateResponse are passed as username and password respectively to the IXLogin.authenticate() method. This method is responsible for interacting with the ticketing service - it validates the ticket provided and responds back with an instance of IXAuthenticateResponse (see Interfaces section) to Capital Manager. If the ticket establishes authenticity, then Capital Manager communicates the success to the application. Otherwise, the corresponding error message, retrieved from the IXAuthenticateResponse object, is logged in the Capital Manager log.

User Authentication Base Interfaces

IXBaseLogin

```
public interface IXBaseLogin extends IXPlugin{
      IXBaseAuthenticationResponse authenticate(String username, String
      password);
}
```

This interface should not be implemented directly. Instead IXLogin should be implemented for server side plugin and IXClientLogin should be implemented for Capital Client authentication plugin.

IXBaseAuthenticationResponse

```
public interface IXBaseAuthenticationResponse{
      String getErrorMessage();
      boolean isAuthenticUser();
}
```

This interface should not be implemented directly. Instead IXAuthenticationResponse should be implemented for server side plugin and IXClientAuthenticationResponse should be implemented for Capital Client authentication plugin.

The method getErrorMessage() returns the error message if the login fails and a success message if login is successful. If the login fails, this message is logged in the Capital Manager or Capital Application log.

The method isAuthenticUser() returns true if the given user name and password have been successfully authenticated. Otherwise, it returns false.

User Authentication Interfaces

IXLogin

```
public interface IXLogin extends IXBaseLogin{
      IXAuthenticateResponse authenticate(String username, String password);
}
```

This is the interface to be implemented by the application which aims for pluggable authentication on server side i.e. Capital Manager.

The implementation of the authenticate() method should check whether the provided user credentials are valid and send a response accordingly to Capital Manager as IXAuthenticateResponse object.

IXAuthenticationResponse

```
public interface IXAuthenticateResponse {
      String getCHSUserName();
}
```

This is the interface which describes the response format from the authentication plugin to Capital.

The method getCHSUserName() will give the corresponding Capital user name for the given user name provided to IXLogin.

IXClientLogin

```
public interface IXClientLogin extends IXBaseLogin{
      IXClientAuthenticationResponse authenticate(String username, String
      password);
}
```

This is the interface to be implemented by the application which aims for pluggable client side authentication.

The implementation of the authenticate() method should check whether the provided user credentials are valid (validate from an external system like LDAP, Site Minder etc.) and send a response accordingly to Capital Client application as `IXClientAuthenticationResponse` object.

IXClientAuthenticationResponse

```
public interface IXClientAuthenticationResponse extends
IXBaseAuthenticationResponse
{
      String getIdentifier();
      String getTicket();
```

```
}
```

This is the interface which describes the response format from the authentication client plugin to Capital.

The method getIdentifier() returns mapped identifier from the plugin to Capital Client Application.

The method getTicket() returns the ticket to Capital Client Application.

User Authentication Plugin Discovery

IXLogin plugin (Capital Manager side plugin) is searched using regular plugin discovery. It is searched in following order of preference

- Command line argument ***pluginfolder***
- Environment variable CAPITAL_PLUGINS_FOLDER / CHS_PLUGINS_FOLDER.
- "$CAPITAL_HOME\plugins" directory where $CAPITAL_HOME is the Capital installation directory.

IXClientLogin plugin (Capital Client side plugin for Ticket Based Authentication) is searched in following order of preference.

- Command line argument ***clientloginpluginpath***
- Environment variable CAPITAL_CLIENT_LOGIN_PLUGINS_FOLDER
- "$CAPITAL_HOME\clientloginplugins" directory where $CAPITAL_HOME is the Capital installation directory.
- If all of the above are missing then no client authentication plugins are used.

User Authentication Examples

In this section, we shall see the code snippets for example of SimpleLDAPLogin.java (server side plugin) and SimpleLDAPClientAuthenticationLogin.java (a client side plugin).These are Java plugins which take the user login credentials from Capital and validate them against an external system and return a valid Capital User for logging into a Capital application.

The complete examples are available in
$CAPITAL_HOME\doc\plugin\examples\Java\src\com\example\plugin\login.

Java class

Client Plugin:

```
public class SimpleLDAPClientAuthenticationLogin implements IXClientLogin
{
        ……
```

```
public IXClientAuthenticationResponse authenticate(final String username,
final String password)
{
        .......
        return new IXClientAuthenticationResponse()
        {
                @Override public String getIdentifier()
                {
                        return " ";
                }
                @Override public String getTicket()
                {
                        return encryptString(m_userName + "@" + m_password);
                }
.       ......
        }

}

}
```

The example above implements the IXClientLogin interface.

Capital client will invoke the “authenticate” method passing it the user typed in “username” and “password”.Implementation of authenticate method should validate this user name and password against external authentication system (e.g. LDAP, Site Minder, etc.). Once the validation is successful it returns the identifier and the ticket via IXClientAuthenticationResponse object.

Server Plugin :

```
public class SimpleLDAPLogin implements IXLogin
{
        ......
        public IXAuthenticationResponse authenticate(final String username, final
        String password)
        {               .......
        return new IXAuthenticationResponse()
        {
                public String getCHSUserName()
                {
                        return m_userName;
                }.
        ......
        }

        }
```

}

The example above implements the IXLogin interface.

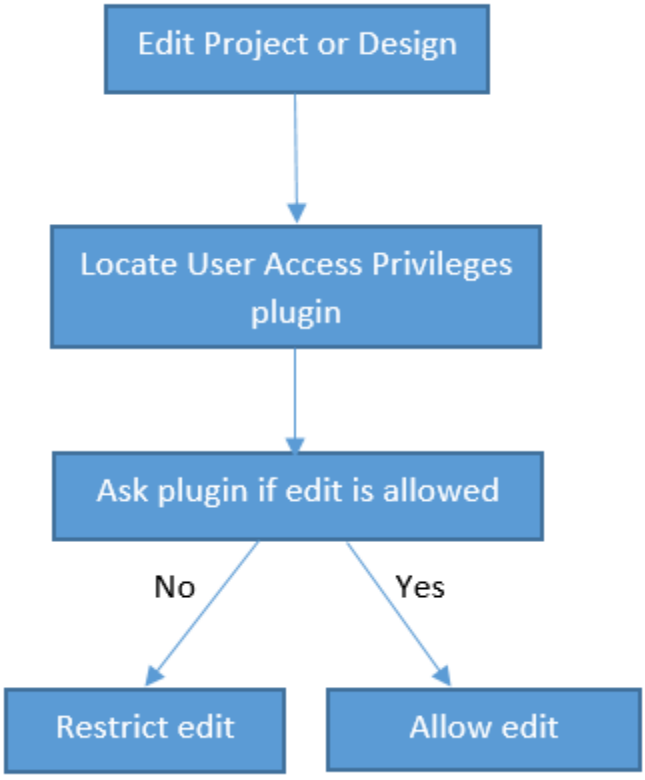
Capital Manager will invoke the “authenticate” method passing it the “username” and “password” passed to it. Implementation of authenticate method should validate this user name and password against external authentication system (e.g. LDAP, Site Minder, etc.). Once the validation is successful it returns the mapped Capital user via IXAuthenticationResponse object (using getCHSUserName() method).

Delegated User Access privileges

Prior to using the delegated user access privileges, the following set-up tasks are required:

- A user access plugin should be written, say Useraccess.jar. The plugin will implement the interface IXUserAccessibilityEvaluator (see Interfaces section). This plugin can be written in Java.
- This jar (useraccess.jar) should be deployed in the plugins directory of the client machine where the Capital application runs.
- Capital Applications is restarted after deploying the plugin. On restart, Capital Applications will scan the plugins directory and load the plugin and recognises that user access should be delegated. Please note that if more than one user access control plugins have been deployed, then all of the plugins are ignored and user access is attempted using Capital’s in-built mechanism (Domain).

During the access of Design or Project, the events shown in the illustration below will take place:



Here is the main flow of events for Project.

1. An application user launches Capital Design or Capital Project.
2. While loading the Project Capital restricts the Project access based on Domain defined on it.
3. The loaded Project further gets filtered by the custom user access privileges plugin.
4. The user can access Projects on which it has a read or write permission (allowed by Domain and custom plugin).

Here is the main flow of events for Designs.

1. An application user launches Capital Design or Capital Project.
2. Selects a Design to open its diagram or tries to edit the basic Design's attributes.
3. Design access will be validated by the Domain defined on it followed by the custom plugin.
4. The user can access Design's content on which it has a read or write permission (allowed by Domain and custom plugin).

User Access privileges Interfaces

IXUserAccessibilityEvaluation

```
public interface IXUserAccessibilityEvaluation<O extends IXEvaluator> extends
IXEvaluation<O>
{
}
```

This interface should be implemented to create the user access privileges plugin. This plugin will be automatically picked up by Capital applications to control user's access.
The Interface extends **`IXEvaluation.`**
The plugin should implement the method getEvaluators() defined in **IXEvaluation** to provide the Projects and Designs custom access privileges(which implements **IXUserAccessibilityEvaluator)** to Capital applications.

IXUserAccessibilityEvaluator

```
public interface IXUserAccessibilityEvaluator<O extends IXObject> extends IXEvaluator
{
   void apply(IXUser currentUser, Collection<IXUserAccessibilityResult<O>> results);
}
```

The apply method will be called by Capital applications to get the access controls of the current user for Project and Designs. The access privileges should be set into the accessibility results (**IXUserAccessibilityResult).** The Project and Design can have the following status - read access, write access, no access or unknown.

IXUserAccessibilityResult

```
public interface IXUserAccessibilityResult<O extends IXObject>
{
    O getObject();

    UserAccessibility getAccessibility();

    void setAccessibility(UserAccessibility accessibility);

    String getMessage();

    void setMessage(String message);
}
```

User should not implement this interface.

Objects of type **IXUserAccessibilityResult** will be passed to the IXUserAccessibilityEvaluator apply method by Capital application. The **getObject** method will return the object under access privileges (either IXProject or IXDesign depending on the type of **IXUserAccessibilityEvaluator**). Plugin will set the accessibility of this object by setting **UserAccessibility** to **READONLY**, **WRITE**,**NOACCESS** or **UNKNOWN** using **setAccessibility** method.

If there is any specific message related to access privileges that needs to be returned to Capital, the plugin can set it using **setMessage** method. It will be logged in the application log file.

UserAccessibility

```
public enum UserAccessibility
{
    UNKNOWN,
    READONLY,
    WRITE,
    NOACCESS
}
```

This will be used by plugin to indicate the access privileges of the Design or Project back to Capital application.

User Access privileges Plugin working flow

IXUserAccessibilityEvaluation plugin (Client side plugin) is searched using regular plugin discovery in "$CAPITAL_HOME\plugins" directory where $CAPITAL_HOME is the Capital installation directory.

Once this plugin is found its getEvaluators will be called to collect the Project and Design user access privileges plugin. There should be only one Project access privileges plugin and only one Design access privileges plugin, otherwise Capital will not use these plugins.

When user accesses Designs or loads the Project, these plugins will be applied by Capital to restrict its access to client.

User Access privileges Examples

In this section, we shall see the code snippets for example of **UserAccessRestrictionPlugin**.java (Client side plugin), **ProjectAccessFilter**.java (a project filter) and **DesignAccessFilter**.java (a design filter).This plugin will validate user access privileges on Design or Project based on a user property ("UserAccess") value.

The complete examples are available in
$CAPITAL_HOME\doc\plugin\examples\Java\src\com\example\plugin\useraccess.

Java class

User Access Plugin:

```
public class UserAccessRestrictionPlugin implements IXUserAccessibilityEvaluation
<IXEvaluator>
{

        public IXEvaluator[] getEvaluators()
        {
                return new IXEvaluator[]{new ProjectAccessFilter(),
                                        new     DesignAccessFilter()};
        }

}
```

The example above implements the IXUserAccessibilityEvaluation interface.

Capital client will invoke the "getEvaluators" to retrieve the ProjectAccessFilter and DesignAccessFilter. These user access privileges are then applied on Designs and Projects to restrict their access.

User Access Filters :

```
public class ProjectAccessFilter implements IXUserAccessibilityEvaluator<IXProject>
{

        public void apply(IXUser currentUser,
```

```
                Collection<IXUserAccessibilityResult<IXProject>> results)
        {
                if (results.isEmpty()) {
                        return;
                }
                for (IXUserAccessibilityResult<IXProject> access : results) {
                        Set<IXValue> props = access.getObject().getProperties();
                        String value = null;
                        for (IXValue prop : props) {
                                if ("UserAccess".equalsIgnoreCase(prop.getName())) {
                                        value = prop.getValue();
                                }
                        }
                        if (value == null) {
                                access.setAccessibility(UserAccessibility.WRITE);
                                continue;
                        }

                        if ("Write".equalsIgnoreCase(value)) {
                                access.setAccessibility(UserAccessibility.WRITE);
                        }
                        else if ("Read".equalsIgnoreCase(value)) {
                                access.setAccessibility(UserAccessibility.READONLY);
                        }
                        else {
                                access.setAccessibility(UserAccessibility.NOACCESS);
                        }
                }
        }
}
```

The example above implements the **IXUserAccessibilityEvaluator** interface.

The user access privileges plugin of type IXProject retrieves the value of a user property called "UserAccess" and based on it sets the access privileges for the Project.

```
public class DesignAccessFilter implements IXUserAccessibilityEvaluator<IXDesign>
{

        public void apply(IXUser currentUser,
                        Collection<IXUserAccessibilityResult<IXDesign>> results)
        {
                if (results.isEmpty()) {
                        return;
                }
                for (IXUserAccessibilityResult<IXDesign> access : results) {
                        Set<IXValue> props = access.getObject().getProperties();
```

```
                        String value = null;
                        for (IXValue prop : props) {
                                if ("UserAccess".equalsIgnoreCase(prop.getName())) {
                                        value = prop.getValue();
                                }
                        }
                        if (value == null) {
                                access.setAccessibility(UserAccessibility.WRITE);
                                continue;
                        }

                        if ("Write".equalsIgnoreCase(value)) {
                                access.setAccessibility(UserAccessibility.WRITE);
                        }
                        else if ("Read".equalsIgnoreCase(value)) {
                                access.setAccessibility(UserAccessibility.READONLY);
                        }
                        else
                        {
                                access.setAccessibility(UserAccessibility.NOACCESS);
                        }
                }
        }
}
```

The example above implements the **IXUserAccessibilityEvaluator** interface.

The user access privileges plugin of type **IXDesign** retrieves the value of a user property called "UserAccess" and based on it sets the access privileges for the design.

Using User Authentication and User Access privileges plugin together

User can implement custom authentication plugin to restrict user logging into Capital by verifying user credentials from external system and further restrict the access to Capital Design and Projects by implementing custom user access privileges plugin. By implementing both the plugins the complete security aspect (authentication and content access) can be externalized.

Assumptions

Currently Capital only supports access privileges of IXProject and IXDesign. The Design and Project

access is controlled in conjunction with the Domain defined on them.